

DOMAIN 2 — SYSTEM DESIGN + ARCHITECTURE

1. Functional Decomposition

Interview Expectation

Candidates should break a complex system into:

- Independent services
 - Clear bounded contexts
 - Scalable modules
 - Ownership domains
-

Example: Cart Management + Order Tracking System

Core services:

User Service

Responsibilities:

- Authentication
- User profile
- Session management

Product Service

Responsibilities:

- Product catalog
- Pricing
- Availability

Cart Service

Responsibilities:

- Add/remove item
- Save for later
- Cart persistence

Inventory Service

Responsibilities:

- Stock management
- Reservation logic
- Inventory synchronization

Order Service

Responsibilities:

- Order creation
- Order lifecycle
- Payment coordination

Payment Service

Responsibilities:

- Payment processing
- Refund handling
- Transaction validation

Delivery Service

Responsibilities:

- Shipment tracking
- ETA updates
- Status events

Notification Service

Responsibilities:

- Email
 - SMS
 - Push notifications
-

2. Event Driven Architecture (Kafka)

Interview Expectation

Candidates should:

- Understand asynchronous communication
 - Know event flow
 - Explain Kafka internals
 - Design resilient event systems
-

Kafka Core Concepts

Producer

Publishes events.

Example:

- OrderCreated event

Consumer

Consumes events.

Example:

- Inventory service updates stock

Topic

Logical event channel.

Example:

- order-events
- payment-events

Partition

Provides scalability and parallelism.

Consumer Group

Enables load balancing.

Important Kafka Interview Topics

Exactly Once vs At Least Once

At Least Once:

- Safer
- May create duplicates

Exactly Once:

- More complex
 - Prevents duplicates
-

Idempotent Consumers

Consumers should safely process duplicate events.

Dead Letter Queue (DLQ)

Used when event processing fails repeatedly.

Ordering Guarantees

Ordering only guaranteed within a partition.

Retry Mechanisms

Candidates should discuss:

- Retry topics
 - Backoff strategy
 - Poison message handling
-

Event Flow Example

1. User places order
2. Order Service publishes OrderCreated
3. Inventory Service reserves stock

4. Payment Service processes payment
 5. Delivery Service initiates shipment
 6. Notification Service sends update
-

3. Database Design

Interview Expectation

Candidates should:

- Choose correct DB
 - Explain normalization
 - Explain scalability strategy
 - Discuss consistency trade-offs
-

SQL vs NoSQL

SQL

Use when:

- Strong consistency needed
- Transactions important
- Relational data exists

Examples:

- PostgreSQL
- MySQL

NoSQL

Use when:

- High scalability needed
- Flexible schema
- Massive throughput

Examples:

- Cassandra

- MongoDB
 - DynamoDB
-

CAP Theorem

System can only guarantee two:

- Consistency
 - Availability
 - Partition tolerance
-

Database Scaling

Vertical Scaling

Increase machine size.

Horizontal Scaling

Add more machines.

Sharding

Partition data across nodes.

Examples:

- User ID based
 - Region based
-

4. Scalability & Performance

Important Concepts

Load Balancer

Distributes traffic.

Examples:

- NGINX
- ALB

Caching

Use cases:

- Product catalog
- Session data
- Frequently accessed data

Technologies:

- Redis
 - Memcached
-

Cache Strategies

Cache Aside

Application manages cache.

Write Through

Update DB + cache together.

Write Back

Write cache first.

CDN

Used for:

- Images
 - Static assets
 - Global low latency
-

5. Reliability & Resilience

Interview Expectation

Candidates should know how systems survive failures.

Circuit Breaker

Prevents cascading failures.

Common states:

- Closed
- Open
- Half-open

Tools:

- Resilience4j
 - Hystrix
-

Bulkhead Pattern

Isolates failures.

Rate Limiting

Protects systems from overload.

Algorithms:

- Token bucket
 - Leaky bucket
-

Graceful Degradation

System should partially work even during failures.

Example:

- Rec